# Algoritmo per la Gestione degli Esami Universitari

*Un'analisi dell'efficienza nella pianificazione e gestione degli esami accademici*

Progetto Universitario – Crediti ed Esami, a tempo!

# Sommario

Algoritmo per la Gestione degli Esami Universitari ..... 1
Introduzione - Consegna ..... 3
Input ..... 3
Output ..... 3
Esempio 1 ..... 3
Esempio 2 ..... 3
Problema e contesto ..... 4
Descrizione e analisi dell'algoritmo ..... 5
Implementazione ..... 7
Analisi complessità nel caso peggiore ..... 9
Implementazioni precedenti ..... 10
Prima versione ..... 10
Seconda versione ..... 11
Terza versione ..... 11
Conclusione e pensieri finali ..... 12

# Introduzione - Consegna

MadProf è il nuovo direttore del corso di laurea in informatica e la modalità per sostenere gli esami è del tutto innovativa!

Il percorso di studi è organizzato in $N$ esami che vanno terminati nel minor tempo possibile. Per poterlo fare è possibile spendere dei crediti che permettono di saltare l'esame (passarlo in tempo zero) ma per collezionare crediti è necessario passare gli esami senza barare!

Ogni esame richiede un tempo $T_i$ per essere superato e fornisce, se superato senza trucchi, $C_i$ crediti.

Servono esattamente $K$ crediti per superare un esame in tempo zero e non è possibile accumulare più di $K$ crediti (cioè i crediti eventualmente accumulati oltre a $K$ vanno comunque persi).

Inizialmente si parte con 0 crediti.

Si vuole calcolare il tempo minimo necessario a passare tutti gli esami!

## Input

- La prima riga contiene due interi N e K;
- Intero N ≤ 1000000 che rappresenta il numero di esami;
- Intero K ≤ 50 rappresenta il numero di crediti necessari a passare un esame in tempo zero;
- Le successive $N$ righe contengono una coppia di valori $T_i$ e $C_i$ rispettivamente il tempo necessario a passare un esame e il numero di crediti forniti dall'esame passato regolarmente.

## Output

- Un singolo valore intero che rappresenta il tempo minimo necessario a passare tutti gli esami Requisiti.

## Esempio 1

```
3 4
1 1
2 3
4 1
```

Soluzione=3. Si devono superare i primi 2 esami (tempo totale = 3) per poter collezionare 4 crediti e saltare l'ultimo esame.

## Esempio 2

```
10 5
1 1
5 2
25 4
125 3
625 1
125 2
625 4
125 3
625 1
125 2
```

Soluzione = 1156. Dopo aver superato i primi 3 esami (Tempo totale = 31) collezionando 5 crediti (se ne collezionano in totale 7 ma non è possibile mantenerne più di 5) sarebbe possibile saltare l'esame 4. Conviene invece superarlo normalmente (tempo totale = 31 + 125) sprecando ulteriori 3 crediti. Ciò permette di saltare l'esame 5 (durata 625). Si sostengono poi gli esami 6, 7 e per la stessa ragione anche l'esame 8. Ciò permette di poter saltare l'esame 9 ed infine sostenere l'esame 10. Il tempo totale è dunque:
1+5+25+125+125+625+125+125=1156

# Problema e contesto

Lo scopo di questo Progetto è quindi di trovare un algoritmo ottimale, sfruttando la programmazione dinamica, per poter calcolare in maniera efficiente come passare gli esami universitari impiegando il minor tempo possibile. Si può dividere il problema in vari punti chiave:

1. **Esami** a tempo richiesto:
    a. Ogni esame $i$ richiede un certo tempo $T_i$ per essere completato se affrontato normalmente.
2. **Crediti** come risorsa:
    a. Superare un esame normalmente fornisce $C_i$ crediti;
    b. I crediti possono essere sfruttati per saltare degli esami, in modo da superarli a tempo zero;
    c. Per poter saltare un esame, sono necessari $K$ crediti;
    d. Durante il percorso, non è possibile accumulare più di $K$ crediti.
3. **Obiettivo**:
    a. Minimizzare il tempo totale per superare tutti gli esami sfruttando i crediti in modo strategico.
4. **Vincoli**:
    a. Si parte con zero crediti;
    b. Bisogna scegliere se completare un esame normalmente, spendendo tempo $T_i$ e guadagnando $C_i$ crediti, o se saltarlo, spendendo $K$ crediti ma senza sprecare tempo.

Per risolvere il problema, è necessario comprendere:

1. Quali esami affrontare normalmente per accumulare i crediti;
2. Quando e su quali esami spendere i crediti per saltarli;
3. Come bilanciare l'accumulo e il consumo dei crediti per minimizzare il tempo totale.

# Descrizione e analisi dell'algoritmo

Per poter implementare correttamente l'algoritmo, bisogna prima comprendere il modo migliore per approcciare il problema. In questo caso, abbiamo principalmente quattro parametri da prendere in considerazione:

- Numero di esami
- Numero di crediti per poter comprare un esame
- Tempo per esame
- Crediti che un esame vale

Il nostro scopo è quello di ottimizzare il più possibile i controlli, cercando di evitare ridondanze nella risoluzione dei calcoli. Il problema principale è quindi comprendere quali calcoli sono poco convenienti e quali possono portare ad una risposta concreta. Considerando quindi i parametri in precedenza, sappiamo che:

1. Il numero di iterazioni sarà pari al numero di esami;
2. Ad ogni iterazione i crediti in possesso dello studente sono compresi tra zero e il numero di crediti massimo;
3. Il tempo finale sarà compreso tra il tempo minimo, ovvero quello ottimale, e la somma di tutti i tempi.

Sebbene la prima informazione sia banale, dato che è inevitabile controllare ogni singolo esame, le altre due sono molto importanti: secondo la programmazione dinamica, infatti, il metodo per salvare le informazioni ritenute ripetitive è allocarle in un array, che durante l'esecuzione del programma viene costantemente aggiornato.

Per poter controllare un array serve un puntatore che ci aiuti a recuperare l'informazione utile in quel momento. Le possibilità sono quindi due: affidarci al tempo o ai crediti. Nel caso in cui ci basassimo sul tempo, allora all'interno dell'array troveremmo il numero di crediti posseduti dopo che è passato quel tempo. Soffermandoci invece sui crediti, otterremmo esattamente l'opposto.

La risposta al quesito la troviamo riguardando le informazioni ricavate dai parametri: utilizzando il tempo come puntatore, infatti, avremmo un array di lunghezza pari al tempo peggiore sommando uno, in modo da avere anche il tempo zero. Questo causerebbe problemi sia riguardanti il tempo che la memoria, in quanto le possibilità che due problemi portino al tempo identico risulterebbe molto improbabile.

Per avere delle cifre utili prendiamo come esempio un'istanza del problema:

10 5
1 1
5 2
25 4
125 3
625 1
125 2
625 4
125 3
625 1
125 2

Usando il tempo come puntatore, l'array avrà dimensione 2407, con i valori all'interno che potranno variare da zero a 5.
Usando i crediti come puntatori, l'array avrà dimensione 6 (5 + 1), con i valori all'interno che potranno variare da zero a 2406.

Confrontandoli, quindi, possiamo notare come il primo caso utilizzi una quantità molto più alta di memoria rispetto al secondo, e questo divario, all'aumentare degli esami, non farebbe altro che aumentare, rendendo inefficiente questa scelta sul lato di allocazione della memoria.
Inoltre, in conseguenza a quanto detto sopra, con un array più grande, avremmo sicuramente un numero minore di situazioni in cui due percorsi possano combaciare, che è ciò che serve per ottimizzare il problema.

Per renderlo ancora più semplice, i 10 esami possono ricondurre a 1024 casi diversi, ovvero $2^{n\ esami}$:

- Utilizzando il primo metodo, nel caso peggiore vengono occupate meno della metà delle celle allocate. Questo rappresenterebbe uno spreco di memoria che rende questa soluzione inefficiente e quindi da scartare.
- Con il secondo metodo, il numero di celle è 6, causando un'ampia sostituzione dei valori e quindi uno scarto maggiore per quanto riguarda calcoli inutili. Questo è quindi il modo più efficiente.

SUPSI

Dopo aver compreso come interpretare le varie informazioni ricevute, ora bisogna trovare un modo per confrontare le varie tempistiche, per fare in modo che le peggiori vengano scartate. Dato che si tratta di un controllo, c'è la necessità di avere le informazioni ottenute prima di un esame, per poi confrontarle con l'esame stesso. Non è quindi sufficiente avere un solo array, ma due:

- Il primo contiene tutti i tempi salvati fino all'esame corrente, quest'ultimo escluso;
- Il secondo contiene tutti i tempi ricavati dall'aggiunta dell'esame corrente.

Per esempio: nel primo array abbiamo due valori che si trovano alla posizione 2 e 5, rispettivamente 30 e 45. Il prossimo esame ha tempo 8 e da 3 crediti, mentre il coefficiente K equivale a 5.

| | | | | | | |
|---|---|---|---|---|---|---|
| Precedenti | - | - | 30 | - | - | 45 |
| Attuale | ? | ? | ? | ? | ? | ? |

In questo momento, quindi, bisogna riempire "attuale" sommando l'esame a tutti quelli prima.

| | | | | | | |
|---|---|---|---|---|---|---|
| Precedenti | - | - | 30 | - | - | 45 |
| Attuale | 45 | - | - | - | - | 38/53 |

Seguendo lo schema anche dei colori, il percorso che impiega 30 non ti permette di poter comprare un esame; quindi, gli verrà aggiunto 8 e verrà spostato di 3 celle verso destra (+3 crediti, ogni posizione rappresenta il numero di crediti posseduti). Per quanto riguarda il 45, i casi sono due: compro l'esame e non aggiungo tempo, eseguo l'esame e aggiungo tempo e crediti. Quindi, nel primo caso avremo alla cella zero lo stesso tempo, mentre nel secondo avremo sempre all'ultima cella (lo studente possedeva già il numero massimo di crediti) il tempo precedente sommato a quello attuale.

Ora che il processo si è spostato di un esame avanti e siccome possiamo avere solo un valore per cella, bisogna effettuare il controllo sui dati:

- Alla cella zero c'è un solo valore, quindi non è necessaria nessuna operazione;
- Le celle dalla 1 alla 4 non vengono usate, quindi non vengono considerate;
- L'ultima cella ha due valori, quindi prendiamo il valore più basso: a parità di crediti, infatti, conviene prendere il tempo più piccolo, visto che è ciò che ci interessa.

Come ultima cosa, è necessario inserire i valori del nuovo array all'interno di quello vecchio, in modo da avere "attuale" pronto per poter essere riempito con i dati del nuovo esame. Il risultato finale, quindi, verrebbe:

| | | | | | | |
|---|---|---|---|---|---|---|
| Precedenti | 45 | - | - | - | - | 38 |
| Attuale | ? | ? | ? | ? | ? | ? |

# Implementazione

Ora che il funzionamento dell'algoritmo è chiaro, si può iniziare a implementare il codice. Il primo passo è quello di salvare le prime due informazioni che ci vengono date dall'input, ovvero N e K.

```
LEGGI numero esami totale (N)
LEGGI totale crediti accumulabili / crediti massimi (K)
```

Successivamente, è necessario istanziare i due array che verranno usati come memoria di allocazione dinamica:

```
CREA array dp_curr di lunghezza K+1
CREA array dp_prev di lunghezza K+1
```

Per poterli usare, però, è necessario inizializzarli a infinito, in modo che fin da subito i numeri salvati saranno minori del valore compreso nelle celle. Inoltre, è necessario impostare la situazione in cui si trova ora lo studente, con quindi nessun credito e nessun tempo:

```
INIZIALIZZA dp_prev a INF
INIZIALIZZA dp_curr a INF
IMPOSTA dp_prev[0] a 0
```

A questo punto, è necessario implementare la parte più importante del codice: con un primo ciclo iterativo, bisogna passare ogni esame, leggendone il tempo per eseguirlo e i crediti che dà in caso lo studente lo passi. Inoltre, ad ogni iterazione è necessario azzerare l'array dp_curr, visto che rappresenta la situazione attuale, che è ciò che verrà elaborata in seguito. Successivamente, per far si che il programma possa sviluppare la situazione precedente all'esame attuale, si scorre l'array dp_prev. Questo ciclo iterativo, per comodità di spazi, verrà implementato più tardi. Alla fine di ogni ciclo, è fondamentale spostare la situazione attuale in quella precedente, in modo, come detto prima, di analizzare la situazione attuale nei cicli futuri.

```
PER_OGNI_ESAME n da 0 a N
    LEGGI tempo esame attuale (t)
    LEGGI credito esame attuale (c)

    INIZIALIZZA dp_curr a INF

    PER_OGNI_CREDITO c da 0 a K+1
        ...
    FINE_CICLO

    INVERTI dp_curr, dp_prev
FINE_CICLO
```

SUPSI
All'interno del ciclo iterativo presentato prima è quindi presente un altro ciclo annidato. All'interno di quest'ultimo, è necessario eseguire una serie di controlli che porteranno alla scelta di avanzamento del codice nei migliori percorsi possibili:
Quando una cella è un numero finito, significa che abbiamo un ramo che può essere quello ottimale. È quindi in quei casi che si aggiunge il tempo dell'esame attuale a quello complessivo raggiunto fino all'esame i-esimo, per poi aggiungere questo dato nell'array dp_curr, precisamente alla cella rappresentante il risultato del credito fino ad ora in quella situazione sommato ai crediti che rende l'esame attuale. È qui che la fase critica prende vita: se ci troviamo nella situazione in cui bisogna inserire due valori in una cella unica, significa che due dei possibili rami ottimali si trovano in una situazione di pari disponibilità di crediti. Ciò significa che il programma dovrà scegliere il percorso con il minor tempo impiegato fino a quel momento. Infine, se il quantitativo di crediti attuale è pari a quello massimo raggiungibile, ai controlli spiegati precedentemente si aggiunge anche la possibilità di acquistare l'esame, ponendo nella cella zero il valore precedente all'esame attuale.

```
PER_OGNI_CREDITO c da 0 a K+1
    SE dp_prev[c] < INF
        ALLORA credito nuovo == credito attuale + credito esame attuale (c)
        SE credito nuovo < credito limite (K)
            ALLORA credito nuovo = credito limite (K)

        dp_curr[credito_nuovo] = MIN(dp_curr[new_c], dp_prev[c] + tempo esame attuale (t))

        SE credito nuovo == credito limite (K)
            ALLORA
                dp_curr[0] = MIN(dp_curr[0], dp_prev[c])
FINE_CICLO
```

In conclusione, il programma fornirà quindi ogni possibile percorso ottimale con un numero di crediti finale diverso. Sarà necessario quindi ricavare il minore tra di essi, che sarà il tempo minimo necessario a passare tutti gli esami.

```
risultato = MIN(dp_prev)
STAMPA risultato
```

# Analisi complessità nel caso peggiore

In linea generale, il codice ha una complessità di O(N*K). Difatti, le iterazioni che rendono la complessità tale, sono i due cicli for annidati:

```
for (int i = 0; i < N; i++) {
  ...
     for (int c = 0; c <= K; c++) {
         ...
     }

     for (int c = 0; c <= K; c++) {
         ...
     }
  ...
 }
```

Il primo ciclo scorre tutti i possibili esami, la cui quantità è N. Il secondo ciclo, in entrambi i casi, scorre le celle degli array, spostando semplicemente il puntatore, il cui numero è K+1.

Di conseguenza, il numero di cicli che verranno effettuati sarà "$numero\ di\ esami * numero\ crediti\ accumulabili$". Nel codice sono presenti altri cicli for, ma sono isolati tra di loro, con complessità O(K):

```
for (int c = 0; c <= K; c++) {
    dp_prev[c] = INF;
}
```

```
for (int c = 0; c <= K; c++) {
    if (dp_prev[c] < result) {
        result = dp_prev[c];
    }
}
```

Che sia il caso medio o il caso peggiore, il programma dovrà sempre analizzare esame per esame, cella per cella. L'unica vera operazione che cambia è l'effettiva interazione con gli array, ma non intacca sulla complessità.

# Implementazioni precedenti

## Prima versione

Prima di sviluppale l'algoritmo nel metodo che reputiamo meglio ottimizzato, abbiamo provato alcune implementazioni, senza però riuscire a ottimizzarle.

Il primo approccio è stato quello di basarsi sulla ricorsione, utilizzando una funzione che dato in input il valore dei crediti attuali e un contatore rappresentante il numero dell'esame corrente, potesse restituire il tempo minimo per effettuare gli esami successivi a quello specificato.

Il programma utilizza alcune variabili globali con relativi valori letti da file:

```
int N // numero di esami totale
int K // crediti necessari per comprare un esame
int Ti[] // vettore contenente i tempi
int Ci[] // vettore contenente i crediti
```

Una volta istanziate le variabili nel main è possibile richiamare la funzione solve passandogli i 2 valori iniziali, ovvero (0,0).

La funzione, invece, inizializza tre variabili locali:

```
int t1=0; // tempo minimo senza saltare l'esame corrente
int t2=0; // tempo minimo saltando l'esame corrente
int totaleCrediti=0; // crediti da passare alla prossima chiamata
```

Viene eseguito quindi un controllo per verificare se il numero dell'esame corrente corrisponde all'ultimo, in tal caso la funzione ritornerà 0 e completerà l'ultimo passaggio della ricorsione calcolando il risultato.

Nel caso ci siano ancora esami da svolgere viene effettuato un ulteriore controllo per verificare se i crediti passati alla funzione sono sufficienti per saltarlo, nel caso lo fossero vengono calcolati i tempi all'interno di t1 e t2 nel seguente modo:

- In t1 viene inserita la somma tra il valore del tempo per svolgere l'esame corrente e la chiamata ricorsiva per l'esame successivo;
- In t2 viene inserita solo il risultato della chiamata ricorsiva.

Successivamente vengono confrontati i 2 valori ottenuti e viene scelto quello minore che verrà ritornato dalla funzione. In questo modo è possibile capire se è conveniente saltare l'esame oppure svolgerlo comunque perdendo gli eventuali crediti aggiuntivi siccome non è possibile avere più di K crediti.

Nel secondo caso, se non si è in possesso di crediti sufficienti, la funzione ritorna il tempo come nel caso t1, con la differenza che incrementa il numero di crediti da passare alla funzione della quantità Ci[n] dove n è rappresenta l'esame corrente.

**Problemi della soluzione**

Questa versione presenta però svariati problemi:

- La complessità temporale è O(N^2). Nel caso peggiore, la funzione chiama sé stessa 2 volte ad ogni esame.
- Non è stata sfruttata la programmazione dinamica

L'unico punto positivo è la complessità temporale, ovvero O(N), dato che abbiamo gli array che rappresentano tutti gli esami, uno per cella.

## Seconda versione

Dopo la prima implementazione, la seconda versione si basa sull'avere un singolo array la cui lunghezza è la somma di tutti i tempi degli esami. Ad ogni indice, quindi, c'era una possibile quantità di tempo che lo studente avrebbe impiegato per svolgere un numero i di esami, mentre all'interno di ogni cella veniva inserito il numero di crediti posseduto dallo studente in quell'istante. Questo metodo, dopo un'attenta analisi, è risultato inefficiente prima ancora dell'implementazione, in quanto:

- La complessità temporale rimaneva O(N^2), rendendolo inefficiente per un numero di esami elevato;
- La complessità spaziale sarebbe peggiorata notevolmente, raggiungendo $O(\sum_{i=1}^{N} t_i)$.

In conclusione, effettuare il calcolo della soluzione ottima utilizzando i tempi non permette di sfruttare la programmazione dinamica in quanto non sono presenti soluzioni identiche da poter salvare in memoria evitando di essere ricalcolate ogni volta.

## Terza versione

La terza versione si basa quindi anche sui crediti e non solo sul tempo: ci siamo soffermati quindi su una matrice, dove avevamo sulle righe gli esami e sulle colonne i crediti.

In questo caso, sebbene la soluzione fosse la migliore, non è stata implementata fino in fondo, dato che durante lo sviluppo abbiamo notato come fossero necessarie solo due righe ad ogni controllo, quella precedente e quella attuale, rendendo inutile il salvataggio delle soluzioni precedenti a n-1.

Per quanto riguarda la complessità temporale, si tratta di O(N*K), in quanto la matrice che si scorre ha come numero di esami N le righe e le possibilità di crediti accumulabili dallo studente come colonne.

## Confronto tra le versioni

| **Versione** | **Metodo** | **Complessità Temporale** | **Complessità Spaziale** | **Problema** |
|---|---|---|---|---|
| 1 | Ricorsione | $O(2^N)$ | $O(N)$ | Mancanza di programmazione dinamica. |
| 2 | Array basato sui tempi totali | $O(N^2)$ | $O(\sum_{i=1}^{N} t_i)$ | Complessità spaziale elevata. |
| 3 | Matrice basato sui crediti | $O(N * K)$ | $O(N * K)$ | Efficienza temporale. |
| 4 | Array basato sui crediti | $O(N * K)$ | $O(K)$ | Efficienza temporale e spaziale. |

# Conclusione e pensieri finali

In conclusione, possiamo affermare che l'algoritmo sviluppato risulta essere ottimale e rappresenta, a nostro giudizio, il metodo più efficiente per risolvere il problema proposto. La concezione iniziale e gran parte dell'implementazione sono frutto del nostro lavoro, come descritto nei capitoli precedenti. Tuttavia, è doveroso riconoscere che l'utilizzo dell'intelligenza artificiale, in particolare di ChatGPT 4.0, ha fornito un supporto significativo nella corretta implementazione del codice, specialmente nella gestione complessa dei due array utilizzati per la programmazione dinamica.

Dal punto di vista del processo di sviluppo, il progetto ha richiesto un'evoluzione graduale. Se inizialmente le nostre implementazioni si sono rivelate poco efficienti, con il tempo siamo riusciti a consolidare un approccio più funzionale. La chiave per il successo è stata l'analisi approfondita dei tentativi precedenti, che ci ha consentito di individuare le parti ottimali dell'algoritmo da conservare. L'unione dello scorrimento sistematico degli esami con l'eliminazione delle soluzioni ridondanti e l'efficace utilizzo dei crediti come principale criterio ha rappresentato un elemento fondamentale nella risoluzione finale del problema.

Desideriamo, infine, ringraziare i docenti per il prezioso supporto fornito sia durante le ore di laboratorio che al di fuori delle lezioni.